Process Scheduling Algorithms ⏱️

Process scheduling is the activity of the operating system's scheduler to select a process from the ready queue and allocate the CPU to it.[1] The goal is to maximize CPU utilization and provide a fair share of the CPU to all processes.[2]

Process scheduling algorithms can be broadly categorized into two types: preemptive and non-preemptive.[3]

Preemptive Scheduling

Preemptive scheduling is a CPU scheduling approach where a currently running process can be interrupted and moved to the ready state.[4] The decision to preempt a process is usually based on a predefined priority, time-slice, or another criterion.[5] This ensures that all processes get a fair share of CPU time and that no single process can monopolize the CPU.[6] It is particularly useful in multi-tasking and real-time operating systems.[7]

- **How it works**:
  - A process is given a time slice to execute.[8] If it doesn't complete within that time, it's interrupted (preempted) and another process is given the CPU.
  - If a higher-priority process arrives, the currently running, lower-priority process is preempted and the CPU is allocated to the new process.[9]
- **Examples**: Round Robin, Shortest Remaining Time First (SRTF), Priority-based scheduling.

Non-Preemptive Scheduling

Non-preemptive scheduling is a CPU scheduling approach where a process, once allocated the CPU, continues to run until it either terminates or voluntarily gives up the CPU.[10] This happens when the process switches to the waiting state (e.g., waiting for I/O).[11] The running process cannot be interrupted by the scheduler.

- **How it works**:
  - A process holds the CPU until it completes its task or blocks itself.
  - The scheduler only dispatches a new process when the current one releases the CPU.
- **Examples**: First-Come, First-Served (FCFS), Shortest Job First (SJF).[12]

Preemption vs. Non-Preemption: Key Differences

Feature	Preemptive	Non-Preemptive
Control	The operating system can forcefully switch the process.	The process itself decides when to release the CPU.
Fairness	More fair, as every process gets a chance to run.	Less fair, as a long-running process can cause starvation.

Feature	Preemptive	Non-Preemptive
Overhead	High, due to context switching between processes.	Low, as context switching is less frequent.
Complexity	More complex to implement.	Simpler to implement.
Use Case	Suitable for time-sharing and real-time systems.	Suitable for batch systems.